



Empresa Operativa Chile

Crear, operar y controlar una empresa chilena a través del tiempo

Contabilidad y tributación explicables · Reglas versionadas por año · Evidencia obligatoria
Sin servidor · Sin cuentas · Sin telemetría · Los datos no salen del dispositivo

Android · APK

Windows · MSI/EXE

Navegador · PWA

CLI · Node 20+

● EMPRESA REAL

Tu contabilidad, con bitácora de auditoría de cada cambio.

● SANDBOX

Empresa ficticia para practicar. Nunca toca la empresa real.

11 · Seguridad

Documentación de sistema · Empresa Operativa Chile

Documento 11 de la serie

Versión del manifiesto 1.4.0 · commit 2b0e006

Análisis del 2026-08-27

Documento técnico generado desde docs/system-documentation/. La fuente es el Markdown del repositorio: si este PDF y el Markdown difieren, manda el Markdown. Esta aplicación no presenta ni paga nada ante el SII y no es asesoría tributaria.

11 · Seguridad

Fuente de verdad de la postura declarada: `SECURITY.md`, que fija el modelo de amenazas, lo que la aplicación garantiza y lo que no, y el canal de reporte. Este documento no lo repite: **verifica en el código** cada garantía que ese documento declara, y añade los controles ausentes que la revisión encontró. Donde ambos hablen de lo mismo, manda `SECURITY.md`.

La conclusión, primero

La superficie de ataque de este sistema es **inusualmente pequeña**, y no por casualidad: no hay servidor, no hay cuentas, no hay red, no hay dependencias de producción y la app de escritorio corre con un solo permiso. Casi todas las categorías de un checklist de seguridad web salen «no aplica», y salen así porque el componente no existe, no porque nadie lo haya mirado.

Lo que queda es un riesgo distinto, y el propio `SECURITY.md` lo ordena bien: **el peligro no es que alguien entre, es que la contabilidad salga**. Subir un respaldo real al repositorio, o publicar un artefacto que calcula mal, hacen más daño que cualquier vulnerabilidad clásica en un sistema sin red.

Lo que esta revisión verificó y confirma: 7 de las 7 garantías de `SECURITY.md` están implementadas como dicen estarlo.

Lo que esta revisión encontró de más: un hueco de CSP en la versión publicada en GitHub Pages, y tres controles ausentes que conviene conocer.

Autenticación, autorización, roles y sesiones

NO IDENTIFICADO , y es una decisión, no un olvido:

Control	Estado	Por qué
Autenticación	No existe	No hay servidor ni cuentas. SECURITY.md : «La aplicación no pide PIN ni contraseña»
Autorización	No existe	No hay múltiples usuarios que separar
Roles y permisos	No existen	Un solo operador por dispositivo
Gestión de sesiones	No existe	No hay sesión: localStorage persiste sin caducidad
Cierre de sesión	No existe	No hay nada que cerrar

La consecuencia, dicha sin adornos: quien tenga el dispositivo desbloqueado tiene la contabilidad. El control de acceso lo aporta el sistema operativo —bloqueo de pantalla, cuenta de usuario, cifrado de disco—, no la aplicación. **SECURITY.md** lo declara en «Qué NO garantiza» y está en el roadmap.

Validación y saneamiento de entradas

Hay dos superficies de entrada y ambas están cubiertas.

Entrada del usuario hacia el motor

Once validaciones, todas lanzando `Error`, listadas en 08 · Flujo de datos. Lo relevante desde el punto de vista de seguridad es que **están en el motor y no en la vista**: la CLI y las pruebas entran por ahí sin pasar por ningún formulario, así que la garantía no depende del HTML.

Salida hacia el DOM — escapado por defecto

`apps/web/src/lib/dom.js` define una plantilla `html` que **escapa toda interpolación** — `&`, `<`, `>`, `"` y `'` — salvo que se marque explícitamente con `raw()`.

```
else rendered = esc(v);
```

Por qué importa aquí: los datos de esta app los escribe el usuario —descripciones de gastos, RUT de proveedores, motivos de reapertura— y después se muestran en tablas y en la bitácora. Escapar por defecto y exigir un `raw()` explícito hace que el caso peligroso sea el que hay que teclear a propósito.

Se auditaron los cinco usos de `innerHTML` del repositorio, que son todos los que hay:

Ubicación	Qué asigna	Veredicto
<code>app.js:127</code>	<code>modeBar() + view.render()</code>	Salida de plantillas <code>html</code>
<code>app.js:152</code>	<code>shell()</code>	Idem
<code>lib/dom.js:121</code>	Plantilla <code>html</code>	Idem
<code>lib/shortcuts.js:126</code>	Resultados del buscador de atajos	Datos del propio catálogo, no del usuario
<code>lib/terms.js:90</code>	Tooltip de un término	<code>esc()</code> explícito en los dos campos

No se encontró ninguna asignación de `innerHTML` con datos del usuario sin escapar. **NO IDENTIFICADO**: no hay `eval`, `new Function`, `document.write` ni `insertAdjacentHTML` en el repositorio.

Cifrado

Dato	En reposo	En tránsito
localStorage	Sin cifrar	No hay tránsito
Espejo JSON de Windows	Sin cifrar	No hay tránsito
RespalDOS exportados	Sin cifrar	Lo que el usuario haga con el archivo
Bitácora	Sin cifrar	No hay tránsito

Nada está cifrado, y está declarado. SECURITY.md : «Los datos no están cifrados. Ni en localStorage ni en el espejo de Windows. Quien tenga acceso al dispositivo desbloqueado tiene acceso a la contabilidad.» La contrapartida es que tampoco viaja a ninguna parte, con lo que desaparece toda la clase de riesgos de transporte, de servidor y de proveedor.

INFERENCIA : en Android, localStorage vive dentro del sandbox de la aplicación, así que otra app sin privilegios de root no debería poder leerlo. Eso lo garantiza el sistema operativo, no este código, y no se verificó en dispositivo.

Gestión de secretos

No hay secretos que gestionar. Ni en el código, ni en la configuración, ni en CI —los seis workflows usan sólo el `GITHUB_TOKEN` implícito—, ni en el dispositivo. Es la consecuencia directa de no tener servicios externos.

El repositorio **defiende activamente** esa propiedad. `security.yml`, trabajo `datos`, falla el build si encuentra:

1. un JSON con la firma `"format": "empresa-operativa-chile/backup"` — un respaldo real commiteado;
2. cualquier `.pfx`, `.p12`, `.pem`, `.key`, `.jks` o `.keystore` versionado;
3. archivos bajo `.local-data/`, `private-data/` o `real-company-data/`.

Se ejecutó la misma comprobación sobre el árbol actual en esta revisión: no hay coincidencias. Ningún secreto, certificado, respaldo real ni credencial en el repositorio.

Registro y auditoría

La bitácora `audit` es el control de seguridad más fuerte que tiene el producto:

Propiedad	Estado
Toda mutación queda registrada	✓ Verificado por prueba
Append-only	✓ No existe ninguna operación de borrado en toda la clase
Guarda quién, cuándo, qué y en qué modo	✓ <code>at</code> , <code>mode</code> , <code>action</code> , <code>detail</code>
Reabrir un período o un ejercicio deja el motivo	✓ Y el CPT anterior, en el caso anual
A prueba de manipulación	✗ Es un JSON local: quien controle el dispositivo puede editarlo
Firmada o encadenada por hash	✗ NO IDENTIFICADO
Con retención o rotación	✗ Crece sin techo

La bitácora prueba lo que pasó **frente a un error propio**, que es su caso de uso real, no frente a un adversario con acceso al disco. Conviene no confundir las dos cosas.

NO IDENTIFICADO : no hay registro de eventos de seguridad, ni detección de intrusiones, ni alertas — no hay servidor donde ponerlos.

Superficie de ataque, componente a componente

Componente	Superficie	Controles	Riesgo residual
<code>server.mjs</code>	HTTP en <code>127.0.0.1:4180</code>	Sólo <code>GET / HEAD</code> ; <code>resolveSafe</code> ; loopback; CSP; <code>nosniff</code> ; <code>no-referrer</code>	<code>HOST=0.0.0.0</code> lo expone a la red sin autenticación
App web	Entrada del usuario al DOM	Escapado por defecto; CSP en servidor y Tauri	Sin CSP en GitHub Pages — ver abajo
Service worker	Caché	Descarta todo lo que no sea del propio origen	Ninguno relevante
Comandos Tauri	4 comandos IPC	<code>safe_mode()</code> ; saneo del nombre de archivo; validación del JSON; <code>core:default</code>	Sólo 1 de los 3 saneos tiene prueba
APK	WebView	Depuración apagada; contenido mixto prohibido; <code>CapacitorHttp</code> apagado	Firmado con la clave de debug
CLI	<code>argv</code> y archivos	Las mismas validaciones del motor	Escribe donde le digan; <code>--datos</code> no se restringe
CI	6 workflows	Acciones fijadas a SHA; permisos mínimos por trabajo; sin secretos	Ninguno relevante
Dependencias	Cero en producción	Gate en CI	La superficie de suministro es el propio Node

El hueco: CSP en la versión publicada

Confirmado leyendo el código. La CSP la ponen dos sitios: la cabecera de `server.mjs` y `tauri.conf.json`. `apps/web/src/index.html` **no lleva** `<meta http-equiv="Content-Security-Policy">` — se buscó `http-equiv` en todo `apps/web/src/` y no hay ninguna coincidencia.

GitHub Pages no permite cabeceras propias. Por lo tanto:

Superficie	¿Tiene CSP?
Servidor local (<code>pnpm app</code>)	✅ Cabecera completa
Windows (Tauri)	✅ <code>app.security.csp</code>
Android (Capacitor)	⚠️ REQUIERE VALIDACIÓN — no se comprobó qué política aplica la WebView sin <code>meta</code>
GitHub Pages	❌ Ninguna

Qué significa en la práctica y qué no. La app sigue sin hacer llamadas de red: no hay `fetch` saliente, no hay recursos de terceros, y el escapado por defecto sigue en pie. La CSP no es lo que impide la fuga; es la **red de seguridad** que convertiría un futuro descuido en un error visible. En Pages esa red no está.

Mitigación disponible: añadir la política como `<meta http-equiv>` en `index.html`. Se registra como recomendación en [15 · Riesgos](#) y **no se aplicó**: este trabajo documenta, no corrige.

Categorías clásicas

Riesgo	Aplica	Estado
Inyección SQL	No	No hay base de datos ni SQL
Inyección de comandos	Parcial	<code>execFileSync</code> con arreglo de argumentos —nunca <code>exec</code> con cadena— en <code>build-all.mjs</code> y <code>chrome.mjs</code> . Sólo build, nunca la app
XSS	Sí	Escapado por defecto; auditados los 5 <code>innerHTML</code> ; sin <code>eval</code>
Path traversal	Sí	<code>resolveSafe</code> con separador, probado en CI con 3 vectores; <code>safe_mode()</code> y saneo de nombre en Rust
CSRF	No	No hay endpoints con efectos ni cookies. <code>form-action 'none'</code> de todos modos
CORS	No	No hay API que compartir. <code>connect-src 'self'</code>
Clickjacking	Sí	<code>frame-ancestors 'self'</code> en el servidor. No en Pages
Deserialización insegura	Bajo	Sólo <code>JSON.parse</code> con <code>try/catch</code> ; <code>importAll</code> valida <code>format</code> y <code>formatVersion</code>
Carga de archivos	Sí	Un <code><input type="file"></code> que sólo lee texto y lo pasa por <code>importAll</code> . No se ejecuta ni se guarda como archivo
Redirección abierta	No	No hay redirecciones
Exposición de información	Sí	Sin telemetría; <code>no-referrer</code> ; enlaces externos con <code>noopener,noreferrer</code>
Dependencias vulnerables	Bajo	Cero de producción. Las de build (Capacitor, Tauri CLI) sólo corren en el equipo de quien compila
Datos personales	Sí	RUT de terceros y montos, en claro, en el dispositivo. Sin cifrar

Controles presentes — las 7 garantías de **SECURITY.md**, verificadas

Garantía declarada	Implementación verificada			
No hay telemetría	Un solo <code>fetch</code> en todo el código, en <code>sw.js</code> , que descarta lo ajeno al origen	✓		
Los datos no salen del dispositivo	<code>localStorage</code> + espejo local; salida sólo por exportación explícita	✓		
Cero dependencias de producción	<code>package.json</code> sin <code>dependencies</code> ; gate en <code>security.yml</code>	✓		
El servidor no queda expuesto	<code>`host = process.env.HOST \</code>	<code>`</code>	<code>'127.0.0.1'</code>	✓ (salvo que se cambie la variable)
La app de Windows no escribe donde quiera	<code>core:default</code> ; 4 comandos que validan argumentos	✓		
El espejo no se corrompe a medias	Temporal + <code>rename</code> en Rust, y también en <code>node-store</code>	✓		
El texto del usuario no inyecta marcado	Plantilla <code>html</code> con escapado por defecto	✓		

Y cuatro controles adicionales que **SECURITY.md** no menciona y esta revisión encontró:

1. **Acciones de CI fijadas a SHA**, con un paso que falla si alguien añade una sin fijar.
2. **CodeQL** con el conjunto `security-and-quality`, en cada push, cada PR y los lunes a las 06:00 UTC.
1. **Verificación del APK por dentro** (`verify-apk.mjs`): abre el ZIP y cuenta vistas, módulos del motor y reglas antes de publicar. Un APK vacío compila perfectamente.
1. **Build reproducible**: CI construye dos veces y falla si `build-info.json` cambia. Si el artefacto no es reproducible, el que se publica no es el que se probó.

Controles ausentes o no comprobados

Con la misma claridad que la sección anterior.

#	Control ausente	Impacto	Severidad
1	CSP en GitHub Pages	La red de seguridad no está en la superficie más expuesta	 Alta
2	Cifrado en reposo	Dispositivo desbloqueado = contabilidad accesible	 Alta (declarada)
3	Firma de código	SmartScreen y Android avisan; el usuario no puede distinguir un binario alterado sin comparar el <code>SHA256SUMS.txt</code>	 Alta (declarada)
4	APK de depuración	Firmado con la clave de debug; no publicable en Play	 Media (declarada)
5	Bloqueo local (PIN)	Sin barrera ante acceso físico	 Media (declarada)
6	Manejo de <code>QuotaExceededError</code>	Si <code>localStorage</code> se llena, la escritura se pierde en silencio	 Media
7	Pruebas de los saneos de Rust	2 de 3 validaciones de <code>lib.rs</code> sin prueba propia	 Media
8	<code>Subresource Integrity</code>	No aplica: no hay recursos externos	 No aplica
9	Escaneo de dependencias (Dependabot)	No se encontró configuración. Con cero dependencias de producción el impacto es bajo, pero las de build no se vigilan	 Baja
10	Bloqueo entre pestañas	Dos pestañas abiertas: la última escritura gana	 Baja

Los puntos 2, 3, 4 y 5 **están declarados** en `SECURITY.md` y en las notas de cada release: son límites conocidos, no descubrimientos. Los puntos 1, 6, 7, 9 y 10 los añade esta revisión.

Qué se hizo y qué no en esta revisión

Se hizo: lectura completa del código de seguridad relevante; verificación por comando de la ausencia de red, de secretos y de CSP; auditoría de los cinco `innerHTML`; revisión de los seis workflows, de las capacidades de Tauri y de la configuración de Capacitor.

No se hizo, y por qué:

- No se ejecutaron pruebas de penetración, escaneos activos ni ataques contra ningún sistema — el encargo lo prohíbe explícitamente.
 - No se analizaron los binarios publicados: `REQUIERE VALIDACIÓN` con el `SHA256SUMS.txt` del release.
 - No se probó el APK en un dispositivo.
 - No se ejecutó CodeQL localmente; sus resultados viven en la pestaña Security del repositorio y `REQUIEREN VALIDACIÓN` allí.
 - **No se corrigió nada.** Los hallazgos van a [15 · Riesgos](#).
-

Reporte de vulnerabilidades

`SECURITY.md` pide abrir un **Security Advisory privado** en vez de una incidencia pública, e indica qué incluir. Declara además fuera de alcance tres cosas que ya están documentadas: binarios sin firmar, datos locales sin cifrar, y que el borrador F29 no cubra todos los códigos del formulario — esto último es una limitación declarada del producto, no un fallo de seguridad.
